

Scanfor Red — Reset-Aware Identity, Trends & Data-Quality Overhaul

Plain-language overview of all 10 phases

Prepared for review before continuing to Phase 5 · Scanfor Red Alert Triage Dashboard

This project is being delivered as ten separate, reviewable phases rather than one large change. Phases 1–4 are implemented, tested, and running live. Phases 5–10 are designed but not yet built. Each phase below gets a one- or two-sentence plain explanation, followed by a slightly more detailed breakdown of what actually changes.

■ Done & verified ■ Planned, not started

Phase 1 — Stable Identity, Real Migrations & Reset-Safe Counters

DONE

In plain terms: Give every alert a permanent "identity" that survives the daily log rollover, switch schema changes to a real migration tool instead of ad-hoc code, and stop treating a daily counter reset as if the problem suddenly got 100x better.

IN MORE DETAIL

- New `AlertSeries` table holds the permanent, editable identity for one alert (owner, ticket, notes, known-issue link) — `AlertEvent` stays an immutable snapshot from one scan.
- Adopted **Alembic** for schema changes, replacing the old pattern of ad-hoc `ALTER TABLE` code that ran on every app startup.
- New `counter_math.py` detects when the source counter has reset (new day, new log file, or count suddenly dropping) and reports the correct "new errors since last scan" instead of a huge negative number.
- Each source file now gets its own observed timestamp instead of one shared "generated time" for the whole folder (real files can be 15–20 minutes apart).
- Added a processing lock and SQLite WAL mode so the background watcher and the manual "Process Now" button can never double-write data.

90 tests passing · verified against real production data and a real copy of the database.

Phase 2 — Prevent False Resolutions & Bad Data

DONE

In plain terms: Don't mark an alert "resolved" just because one scan happened to miss it or a source file broke — and stop silently accepting malformed or duplicate data rows.

IN MORE DETAIL

- Every source file now gets a quality rating each scan: `ok`, `parse_warning`, `stale` (stopped updating), or `parse_failure` (unreadable).
- Every scan gets an overall rating too: `complete`, `partial` (some files missing/broken), or `missing_source` (everything unavailable).
- An alert only becomes "Resolved" after it's been genuinely absent for a configurable grace period (default: 2 scans *and* 20 minutes) — not the instant it's missing once.
- Rows missing required fields (hostname, tenant, etc.) are now rejected and counted instead of silently stored with blank data; duplicate rows within one file are detected and only the first kept.

95 tests passing · verified by simulating missing and stale files against real data.

Phase 3 — Rate-Based Trend Engine

DONE

In plain terms: This is the core fix the whole project exists for — the dashboard's trend math was still using the raw daily counter, so it still broke every midnight even after Phase 1 fixed the underlying data. Now it uses the corrected numbers, a statistically sound baseline, and a much richer vocabulary for describing what's happening.

IN MORE DETAIL

- All trend math (1-hour change, 6-hour slope, acceleration) now runs on the reset-safe numbers from Phase 1, not the raw counter — verified a midnight rollover no longer corrupts the numbers.
- New "robust baseline" uses the statistical median instead of an average, so one freak spike can't drag the definition of "normal" upward.
- Replaced the old 12-state trend vocabulary with 14 states requested by the spec — `accelerating`, `worsening_rapidly`, `spike`, `worsening_steadily`, `slow_growth`, `cooling`, `resolving`, `persistent`, `stable`, `data_unavailable`, and more.
- An alert whose source data is missing/broken (Phase 2) now correctly shows `data_unavailable` instead of a misleading trend guess.
- All existing API field names were kept the same on purpose — only the numbers behind them got more correct.

105 tests passing · ~20 hand-built scenarios verified one by one to nail down the exact classification rules.

Phase 4 — Explainable Change Score

DONE

In plain terms: A single 0–100 number showing how urgently an alert's behavior is changing, plus an honesty check so a score based on almost no history doesn't look as trustworthy as one backed by hours of real data.

IN MORE DETAIL

- Renamed `growth_score` → `change_score` with a genuinely new formula: 30% short-term rate vs. baseline, 25% sustained 1-hour rate vs. baseline, 20% acceleration, 15% persistence, 10% how many hosts share the same error ("multi-VM spread").
- New `change_score_confidence` : separately reports what percentage of that formula was actually backed by real data, so a score built from one component alone is flagged as low-confidence even though the number itself still looks "full scale."
- Verified live: a synthetic alert seen on 5 hosts scored 40 points higher than an identical single-host alert, purely from the multi-VM component.

110 tests passing · verified live through the running dashboard's API.

Phase 5 — Known-Issue & Change Audit History

PLANNED

In plain terms: Keep a record of who changed what about an alert (owner, ticket, notes, known-issue link, severity) and when — plus a safe "preview" before a known-issue rule change gets applied broadly across the dashboard.

IN MORE DETAIL

- New audit/change-history model: who made the change, which field, the old value, the new value, the reason given, and the timestamp.
- Also audits edits to known-issue matching rules and thresholds themselves, not just individual alerts.
- When a known-issue rule changes, a "dry run" shows what would be reclassified before it's actually applied.
- Historical observations are never deleted just because an issue gets reclassified later.

Phase 6 — API Expansion

PLANNED

In plain terms: Add the extra data and endpoints the redesigned dashboard (Phase 7) will need, without breaking anything that already works today.

IN MORE DETAIL

- The main alerts response gains: series ID, raw count, interval delta, rate/hour, rolling rates, baseline, acceleration, reset-detected flag, trend state, change score + confidence + components, red duration, affected VM count, source freshness, data-quality status, observed/ingested time.
- New endpoint for one alert's history — enough data points for a small sparkline chart.
- New "source health" endpoint: expected vs. discovered vs. missing vs. stale files, timestamp skew across files, parse warnings, and the last complete/partial snapshot.
- All of this uses batched queries — never one extra database query per displayed row.

Phase 7 — Dashboard Redesign

PLANNED

In plain terms: Redesign the actual screen around everything the backend now knows — clearer columns, a "change pattern + score" badge, richer expandable detail, summary cards, and better sorting/filtering.

IN MORE DETAIL

- New column set: status, error, host, log/service, current daily total, new since last scan, new errors/hour, change pattern + score, red duration, affected VMs, owner/ticket, actions.
- Replaces the ambiguous "Growth"/"Scan Δ" labels with a plain badge like `Accelerating · 86`.
- Expanded row shows raw & interval delta, rate, 30m/1h/6h/24h comparisons, baseline, acceleration, score breakdown, source file/timestamp, data-quality warnings, a small sparkline, known-issue match reason, and notes/owner/ticket/runbook.
- New summary cards: Needs Attention, Accelerating, Spikes, Persistent, Multi-VM, Stale/Missing Sources.
- Sorting/filtering by change score, trend state, confidence, source freshness, and more.

Phase 8 — Accessibility & Color Contrast

PLANNED

In plain terms: Make sure the dashboard is actually readable for everyone — proper contrast in both light and dark mode, and never relying on color alone to say "this is bad."

IN MORE DETAIL

- Meet WCAG AA contrast (4.5:1 for small text, 3:1 for large text) in both themes.
- Every status/trend badge gets a readable text label (and icon/shape where useful), not just a color.
- Visible keyboard focus states; hover states that don't reduce contrast; muted text stays readable.
- Audit tables, badges, buttons, links, form fields, and tooltips in both themes; keep red, orange, yellow, green, blue, and purple visually distinct from one another.
- Document the final color tokens and their tested contrast ratios.

Phase 9 — Simulation Harness & Remaining Tests

PLANNED

In plain terms: Build a script that generates realistic fake "hours" of scan data (steady, growing, spiking, resetting at midnight, going missing, recovering) so the dashboard's behavior can be demonstrated and proven, not just asserted.

IN MORE DETAIL

- Replay/simulation script generating a sequence of realistic 10-minute snapshots covering: stable, slow growth, acceleration, spike, midnight reset, missing source, and recovery.
- Used to visually demonstrate every dashboard state end-to-end, not just via unit tests.
- Fills in any remaining items from the original 23-item test list not already covered by Phases 1–4 (for example: score confidence under insufficient history, API still working when one alert's trend calculation fails).

Phase 10 — Documentation

PLANNED

In plain terms: Rewrite the README and add a dedicated reference document explaining how identity, resets, and trends actually work now — plus a final completion report tying everything together.

IN MORE DETAIL

- README rewrite covering: data flow, stable identity, daily reset handling, source time vs. ingestion time, the change-score formula, trend-state definitions, source-health rules, the known-issue workflow, migration steps, environment variables, and production deployment notes.
- New standalone `docs/TREND_AND_IDENTITY_MODEL.md` reference document.
- Final completion report: full test count, every migration listed, all changed files summarized, the trend formula explained in plain language, the midnight-reset test result shown, and an honest list of assumptions/thresholds that still need tuning against real production history.